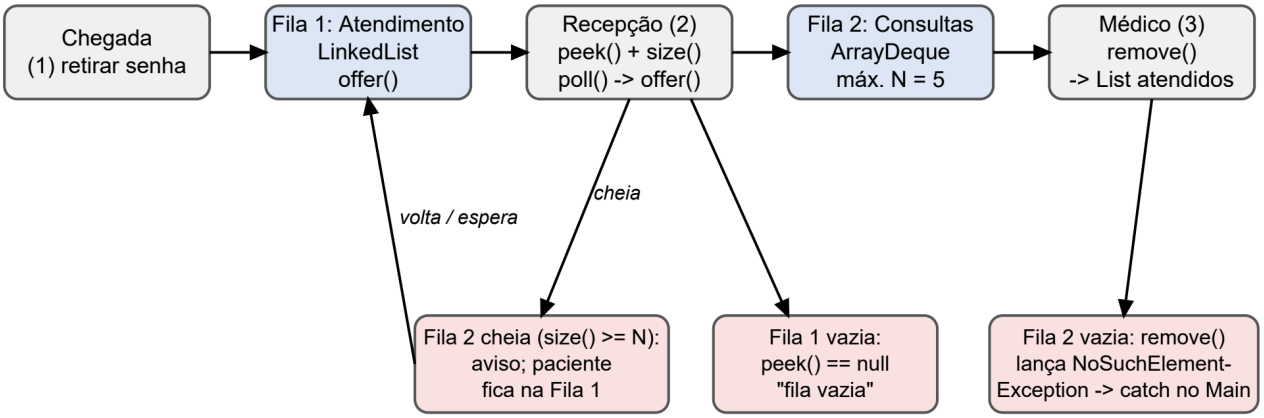
Sistema de Filas de Atendimento e Consultas

Integrantes: Lucas Steves 52420131 · Eduardo Souza 52421070 · Kaique Rebonato 52522487

1. Visão geral e fluxograma

O sistema modela duas filas FIFO independentes. O paciente retira uma senha e entra na **Fila de Atendimento**; ao ser chamado na recepção, passa para a **Fila de Consultas**, que tem limite de N pacientes; quando o médico o chama, vai para uma List<Paciente> de atendidos. O código está dividido em Paciente (dados), ServicoClinica (as duas filas e as regras de negócio), Main (menu com Scanner) e TesteClinica (testes). O caminho do paciente e os desvios tratados:



Caminho do paciente. Caixas azuis são filas; vermelhas são os casos de exceção tratados sem quebrar o programa.

2. Declaração das filas e justificativa da implementação

As filas são declaradas pela interface: Queue<Paciente> filaAtendimento = new LinkedList<>(); e Queue<Paciente> filaConsultas = new ArrayDeque<>();. Isso é polimorfismo: o resto do código só usa métodos de Queue, então trocar a implementação exige mudar apenas uma linha.

Fila 1 – LinkedList. A fila de atendimento não tem limite: pode crescer muito num horário de pico e esvaziar depois. A LinkedList é uma lista duplamente encadeada; inserir no fim e remover do início são O(1) sempre (sem pior caso de redimensionamento), e a memória acompanha o tamanho real da fila, porque cada nó é liberado ao sair. O custo é ~40 bytes extras por nó e pior uso de cache, irrelevantes no volume de uma clínica.

Fila 2 – ArrayDeque. A fila de consultas é pequena e limitada (N = 5). O ArrayDeque é um array circular: com capacidade inicial de 16, ele nunca precisa redimensionar neste cenário, então todas as operações são O(1) reais, com dados contíguos na memória (bom para cache) e sem criar um nó por inserção. Ele também proíbe null, o que garante que poll()/peek() retornarem null signifique sempre “fila vazia”.

Limite da Fila 2. Queue não define capacidade, então o limite é verificado com size() antes de mover o paciente. Se size() >= N, o método lança IllegalStateException, o Main mostra o aviso e o paciente continua na frente da Fila 1, pois só é retirado com poll() depois da verificação.

Bônus – atendimento prioritário (PriorityQueue). Ao iniciar, o sistema pergunta se o modo prioritário deve ser ativado. Nesse caso a Fila 2 é criada como new PriorityQueue<>(ORDEM_PRIORIDADE), com o comparador Comparator.comparing(Paciente::isPrioritario).reversed().thenComparingInt(Paciente::getSenha): prioritários (idosos, gestantes) saem primeiro e, entre iguais, quem tem a menor

senha. Como a fila é declarada como Queue, essa é a única linha que muda; offer, poll, peek, remove e size seguem iguais (polimorfismo na prática). A Fila 1 continua FIFO, porque a prioridade vale só na fila de consultas. Um cuidado: o iterador da PriorityQueue percorre o heap interno, não a ordem de saída, então a listagem (opção 5) ordena uma cópia com o mesmo comparador.

3. Diferença entre os métodos da interface Queue

Cada operação tem duas versões: uma lança exceção quando não consegue executar, e outra retorna um valor especial (false ou null).

Operação	Lança exceção	Valor especial	Diferença na prática
Inserir	add(e)	offer(e)	Numa fila com capacidade limitada e cheia, add lança IllegalStateException e offer retorna false. Em LinkedList/ArrayDeque (sem limite) ambos sempre retornam true.
Remover	remove()	poll()	Com a fila vazia, remove() lança NoSuchElementException; poll() retorna null. Ambos retiram e devolvem o primeiro.
Consultar	element()	peek()	Mostram o primeiro sem remover. Fila vazia: element() lança NoSuchElementException; peek() retorna null.

Onde cada um foi usado. No fluxo normal usamos offer/poll/peek: retirar senha (offer), recepção (peek para ver se há alguém, poll + offer para mover) e a opção 6 (peek nas duas filas). Para demonstrar o outro tipo, chamar para consulta usa remove(): com a Fila 2 vazia ele lança NoSuchElementException, que o Main captura com try/catch e mostra “fila de consultas vazia”. Os testes também verificam add e element diretamente. Em geral, os métodos de valor especial são melhores quando a fila vazia é um caso esperado; os de exceção, quando ela indica um erro.

4. Complexidade (Big-O)

Operação	LinkedList	ArrayDeque	PriorityQueue	Observação
offer / add	O(1)	O(1) amort.	O(log n)	ArrayDeque dobra o array quando enche (raro; com N = 5 nunca). No heap, o novo elemento “sobe” até a posição certa.
poll / remove()	O(1)	O(1)	O(log n)	Lista/array: tira do início. Heap: o último vai para a raiz e “desce”.
peek / element	O(1)	O(1)	O(1)	O primeiro (ou o de maior prioridade, na raiz do heap).
size()	O(1)	O(1)	O(1)	Todas guardam o tamanho; o controle do limite é barato.
listar (iterar)	O(n)	O(n)	O(n log n)	FIFO: o iterador já está em ordem. PriorityQueue: copia e ordena.
contains	O(n)	O(n)	O(n)	Busca linear (usada só nos testes).

A lista de atendidos é um ArrayList: add O(1) amortizado. O relatório calcula o tempo médio de espera (chegada até a chamada do médico) percorrendo os atendidos, O(n).

5. Testes

A classe TesteClinica roda os casos sem biblioteca externa (cada verificação lança AssertionError se falhar) e termina com código de saída 1 se algum falhar. Resultado: **7 de 7 passaram**.

#	Caso	O que é verificado
1	Filas vazias	Recepção com Fila 1 vazia retorna null; peek nas duas filas retorna null; tempo médio = 0.
2	Fila 2 cheia	Com limite 2, o 3º paciente gera aviso, a Fila 2 continua com 2 e ele segue na frente da Fila 1; após uma consulta, ele é encaminhado.
3	Fluxo completo	Um paciente passa por Fila 1 → Fila 2 → atendidos, com hora da consulta registrada.
4	remove() em fila vazia	chamarParaConsulta() lança NoSuchElementException e nada entra em atendidos.
5	Ordem FIFO	Três pacientes saem das duas filas na mesma ordem em que chegaram; senhas sequenciais.
6	Métodos da Queue	poll/peek retornam null e remove/element lançam exceção em fila vazia; add/offer inserem.
7	Bônus: prioridade	Fila 1 segue FIFO; na Fila 2, prioritários saem primeiro e, entre iguais, a menor senha; a listagem mostra a ordem real de chamada.

Como executar (JDK 17+): javac -encoding UTF-8 -d out src/*.java, depois java -cp out Main (sistema) ou java -cp out TesteClinica (testes).